Homework 10

Haoran Xu

2026-11-01

Software note. All forests use the `RLT` package in R. The version that installs from CRAN on this machine is `RLT` 3.2.6, whose *defaults* are extremely randomized trees (`split.gen = "random"`, one random cut point per candidate, `alpha = 0.4`, 90% subsamples). To get ordinary random forests, as the course's newer `RLT` does by default, every fit below sets `split.gen = "best"` (exhaustive CART-style search over the `mtry` candidates), `alpha = 0` (no child-size constraint), `resample.prob = 1` with `replacement = TRUE` (a bootstrap sample of the training size), and `reinforcement = FALSE`. In this version `nmin` is the smallest node size that is still split, so a node with fewer than `nmin` observations becomes terminal; this matches the course description up to one observation. Everything else is left at its default.

```
suppressPackageStartupMessages(library(RLT))
rf <- function(x, y, mtry, nmin, model = "regression", ntrees = 500,
               prob = 1, importance = FALSE, track.obs = FALSE) {
  invisible(capture.output(    # silence the per-fit "not compiled with OpenMP" notice
    fit <- RLT(x, y, model = model, ntrees = ntrees, mtry = mtry, nmin = nmin,
               split.gen = "best", alpha = 0, resample.prob = prob, replacement = TRUE,
               reinforcement = FALSE, importance = importance, track.obs = track.obs)))
  fit
}
```

Question 1 (Bias-variance trade-off simulation)

(a) Simulation

Design. The five predictors are $N_5(\mathbf{0}, \Sigma)$ with $\Sigma_{jj} = 1$ and $\Sigma_{jk} = 0.3$, generated as $\mathbf{z}^\top R$ where $R^\top R = \Sigma$ is the Cholesky factor. The 200 test points are drawn once with seed **432** and held fixed; each of the 100 repetitions draws a fresh training set of 200 observations that is shared by all ten tuning settings. Every forest has 500 trees, each grown on 200 observations drawn with replacement. For setting s and test point i, with $\hat{f}_{sr}(\mathbf{x}_i)$ the prediction in repetition r,

$$\widehat{\text{bias}}_s^2 = \frac{1}{200}\sum_i \left(\bar{\hat{f}}_{s\cdot}(\mathbf{x}_i) - f(\mathbf{x}_i)\right)^2, \qquad \widehat{\text{var}}_s = \frac{1}{200}\sum_i \frac{1}{99}\sum_r \left(\hat{f}_{sr}(\mathbf{x}_i) - \bar{\hat{f}}_{s\cdot}(\mathbf{x}_i)\right)^2,$$

so bias is measured against the true $f(\mathbf{x}) = x_1 + x_3 + x_5$, not against noisy Y. The setting `mtry = 3, nmin = 10` belongs to both comparisons and is fitted twice; the two copies differ only by forest randomness, which gives a feel for the Monte Carlo noise.

```
set.seed(432)
p <- 5; Sigma <- matrix(0.3, p, p); diag(Sigma) <- 1; R <- chol(Sigma)
gen_x <- function(n) matrix(rnorm(n * p), n, p) %*% R
f <- function(x) x[, 1] + x[, 3] + x[, 5]
xtest <- gen_x(200); ftest <- f(xtest)
settings <- rbind(data.frame(par = "nmin", mtry = 3, nmin = c(1, 5, 10, 20, 30)),
                  data.frame(par = "mtry", mtry = 1:5, nmin = 10))
nrep <- 100
preds <- array(NA_real_, c(nrow(settings), nrep, 200))
```

```
for (r in seq_len(nrep)) {
  xtr <- gen_x(200); ytr <- f(xtr) + rnorm(200)
  for (s in seq_len(nrow(settings)))
    preds[s, r, ] <- predict(rf(xtr, ytr, settings$mtry[s], settings$nmin[s]),
                             xtest)$Prediction
}
bv <- function(P) {           # P: nrep x 200 predictions for one setting
  m <- colMeans(P)
  c(bias2 = mean((m - ftest)^2), variance = mean(apply(P, 2, var)))
}
res <- cbind(settings, t(apply(preds, 1, bv)))
res$total <- res$bias2 + res$variance
knitr::kable(res, digits = 3, row.names = FALSE,
             caption = "Averaged squared bias and variance (total = bias$^2$ + variance).")
```

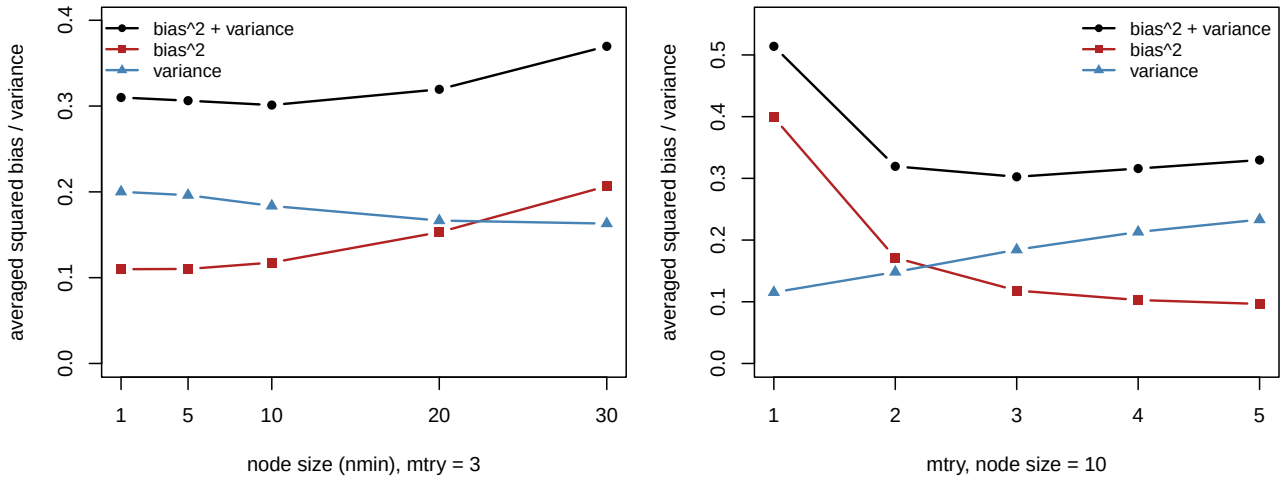
Table 1: Averaged squared bias and variance (total = bias^2 + variance).

par	mtry	nmin	bias2	variance	total
nmin	3	1	0.110	0.200	0.310
nmin	3	5	0.110	0.196	0.306
nmin	3	10	0.118	0.184	0.301
nmin	3	20	0.153	0.167	0.319
nmin	3	30	0.207	0.163	0.370
mtry	1	10	0.399	0.115	0.514
mtry	2	10	0.171	0.148	0.319
mtry	3	10	0.118	0.184	0.303
mtry	4	10	0.103	0.213	0.316
mtry	5	10	0.097	0.233	0.330

```
bvplot <- function(d, xvar, xlab, pos = "topleft") {
  x <- d[[xvar]]
  plot(x, d$total, type = "b", pch = 16, lwd = 1.6, ylim = c(0, max(d$total) * 1.08),
       xlab = xlab, ylab = "averaged squared bias / variance", xaxt = "n")
  axis(1, at = x)
  lines(x, d$bias2, type = "b", pch = 15, lwd = 1.6, col = "firebrick")
  lines(x, d$variance, type = "b", pch = 17, lwd = 1.6, col = "steelblue")
  legend(pos, bty = "n", cex = 0.85, col = c("black", "firebrick", "steelblue"),
         pch = c(16, 15, 17), lwd = 1.6, legend = c("bias^2 + variance", "bias^2",
          ↪  "variance"))
}
```

```
par(mar = c(4, 4.2, 1, 1), cex = 0.85)
bvplot(res[res$par == "nmin", ], "nmin", "node size (nmin), mtry = 3")
par(mar = c(4, 4.2, 1, 1), cex = 0.85)
bvplot(res[res$par == "mtry", ], "mtry", "mtry, node size = 10", pos = "topright")
```

What the numbers show. Node size moves bias and variance in opposite directions: from `nmin = 1` to `30` the variance falls steadily (0.200 to 0.163) while the squared bias almost doubles (0.110 to 0.207), and the sum is smallest at `nmin = 10`, with `nmin = 5` essentially tied. The duplicated setting `mtry = 3, nmin = 10` reproduces to within 0.002, so differences of that order are Monte Carlo noise. For `mtry` the two curves cross: the squared bias drops sharply from 0.399 at `mtry = 1` to about 0.10 at `mtry = 4`–`5`, while the variance *rises* monotonically from 0.115 to 0.233. The total is minimised at `mtry = 3`, the setting where the two curves meet; both `mtry = 1` (dominated by bias) and `mtry = 5` (dominated by variance) are worse.



(a) Node size varied, mtry fixed at 3.

(b) mtry varied, node size fixed at 10.

Figure 1: Squared bias and variance averaged over the 200 fixed test points (100 repetitions, 500 trees, seed 432).

(b) Discussion

Large node size (e.g. 30). With 200 training points and `nmin = 30` a tree has only about 10–15 leaves, so each leaf averages a wide region of a function that keeps changing linearly in three directions. Predictions are pulled toward the leaf mean, which over-smooths the additive signal: the squared bias roughly 1.8 times that of `nmin = 10` in the table. The variance does go down because the leaf averages are over more points, but the saving is small relative to the bias increase, so a large node size *under-fits* in this example. (Averaging 500 bootstrap trees already removes most variance, which is why forests tolerate small node sizes so well.)

Benefits and drawbacks of a large `mtry`. *Benefit:* with `mtry = 5` every split can choose among all predictors, so each tree finds the three signal variables X_1, X_3, X_5 and never wastes a split on the noise variables X_2, X_4. The individual trees are therefore much more accurate, which is why the squared bias falls from 0.399 at `mtry = 1` (where a tree is forced to split on a noise variable 40% of the time and the signal is smeared out) to 0.097 at `mtry = 5`. *Drawback:* a large `mtry` makes the trees similar to one another, because they all pick the same strong variables at nearly the same cut points. The forest variance is roughly $\rho\,\sigma^2_{\text{tree}}$ for a large number of trees, so the higher between-tree correlation ρ limits how much averaging can reduce the variance; this is exactly the monotone rise of the variance curve from 0.115 to 0.233. In this example the variance penalty outweighs the remaining bias gain beyond `mtry = 3`, so the forest with `mtry = 5` (bagging) is worse than `mtry = 3`. A large `mtry` is also costlier per split and, when there are many predictors with a few dominant ones, it can hide weaker predictors that only get a chance to split when the dominant ones are not in the candidate set. The trade-off therefore favours a moderate `mtry`: large enough that the signal variables are usually available, small enough to keep the trees diverse.

Trees grown on 100 observations. Node size controls the number of observations in a leaf, so what matters is the ratio `nmin / (sample size per tree)`. If each tree sees only 100 draws with replacement (about 63 distinct points), a given `nmin` produces leaves that cover twice the fraction of the input space, so the bias of every setting goes up and the "under-fitting" regime starts at a *smaller* `nmin`. At the same time, the subsampling itself injects more randomness into each tree and decorrelates them, which already supplies some of the variance reduction that a larger node size would have given. Both effects push the preferred node size toward *smaller* values. The simulation below repeats the node-size comparison with 100-observation bootstrap samples (`resample.prob = 0.5`, same seed and design).

```
set.seed(432)
xtest2 <- gen_x(200); ftest <- f(xtest2)   # same first draw as above -> same test set
nmins <- c(1, 5, 10, 20, 30)
preds2 <- array(NA_real_, c(length(nmins), nrep, 200))
for (r in seq_len(nrep)) {
  xtr <- gen_x(200); ytr <- f(xtr) + rnorm(200)
  for (s in seq_along(nmins))
```

```
    preds2[s, r, ] <- predict(rf(xtr, ytr, mtry = 3, nmin = nmins[s], prob = 0.5),
                              xtest2)$Prediction
}
res2 <- data.frame(nmin = nmins, t(apply(preds2, 1, bv)))
res2$total <- res2$bias2 + res2$variance
cmp <- data.frame(nmin = nmins, total_200 = res$total[1:5], total_100 = res2$total,
                  bias2_100 = res2$bias2, var_100 = res2$variance)
knitr::kable(cmp, digits = 3, caption = "Node-size comparison with 200 vs 100 bootstrap draws
 ↪  per tree (mtry = 3).")
```

Table 2: Node-size comparison with 200 vs 100 bootstrap draws per tree (mtry = 3).

nmin	total_200	total_100	bias2_100	var_100
1	0.310	0.296	0.167	0.129
5	0.306	0.294	0.168	0.126
10	0.301	0.305	0.188	0.117
20	0.319	0.393	0.281	0.112
30	0.370	0.539	0.424	0.115

With 100 draws per tree the best node size is 5 (versus 10 with 200 draws), the squared bias at every node size is larger than before, and the variance is lower, which is the shift toward smaller node sizes argued above. The differences between neighbouring settings are small relative to the Monte Carlo noise, so I read this as supporting evidence rather than proof.

Question 2 (Tuning random forests for MNIST)

Data and PCA

```
mn <- read.csv("data/mnist-first2000.csv.gz")
keep <- c(1, 6, 7)
tr <- mn[1:1000, ][mn$Digit[1:1000] %in% keep, ]
te <- mn[1001:2000, ][mn$Digit[1001:2000] %in% keep, ]
pca <- prcomp(as.matrix(tr[, -1]), center = TRUE, scale. = FALSE)
Ztr <- pca$x[, 1:20]
Zte <- scale(as.matrix(te[, -1]), center = pca$center, scale = FALSE) %*% pca$rotation[,
 ↪  1:20]
ytr <- factor(tr$Digit); yte <- factor(te$Digit)
c(n_train = nrow(Ztr), n_test = nrow(Zte)); table(train = ytr); table(test = yte)
#> n_train  n_test
#>     327     317
#> train
#>   1   6   7
#> 116  94 117
#> test
#>   1   6   7
#> 104 106 107
round(summary(pca)$importance[3, 20], 3)     # cumulative variance of PC1-20
#> [1] 0.771
```

The training set has 327 images and the test set 317; the three classes are roughly balanced. PCA is fitted on the training pixels only (centering, no scaling, since all pixels are on the same 0–255 scale) and the test images are projected with the training means and loadings. The first 20 components explain 77% of the pixel variance.

OOB machinery. `RLT` stores how often each observation was drawn for each tree in `ObsTrack`. For each training image I average the class probabilities of the trees for which it was out of bag and predict the class with the largest

average; the OOB misclassification rate is the share of images whose OOB prediction is wrong. All forests have 1,000 trees and a bootstrap sample of the full training size.

```
crf <- function(mtry, nmin, importance = FALSE)
  rf(Ztr, ytr, mtry = mtry, nmin = nmin, model = "classification", ntrees = 1000,
     importance = importance, track.obs = TRUE)
oob_pred <- function(fit, x) {
  all <- predict(fit, x)$AllPrediction        # per-tree class-probability matrices
  votes <- matrix(0, nrow(x), fit$nclass)
  for (b in seq_along(all)) {
    o <- fit$ObsTrack[, b] == 0                # out of bag for tree b
    votes[o, ] <- votes[o, ] + all[[b]][o, ]
  }
  factor(fit$ylevels[max.col(votes, ties.method = "first")], levels = fit$ylevels)
}
oob_err <- function(fit) mean(oob_pred(fit, Ztr) != ytr)
```

(a) Default settings

```
set.seed(432)
fit0 <- crf(mtry = max(1, floor(ncol(Ztr) / 3)), nmin = max(1, floor(log(nrow(Ztr)))))
c(mtry = fit0$mtry, nmin = fit0$nmin, oob_error = oob_err(fit0))
#>      mtry      nmin oob_error
#>   6.00000   5.00000   0.03976
```

The package defaults are `mtry = floor(p/3) = 6` and `nmin = floor(log n) = 5`. (The newer course version of `RLT` defaults to `mtry = p/2 = 10` and `nmin = 5`; that setting is part of the grid below.) The default forest already misclassifies only 4% of the training images out of bag, i.e. 13 of 327.

(b) Tuning grid and variable importance

```
grid <- expand.grid(nmin = c(1, 2, 5, 10, 20, 40), mtry = c(1, 2, 4, 6, 10, 15, 20))
set.seed(432)
grid$oob <- sapply(seq_len(nrow(grid)),
                   function(k) oob_err(crf(grid$mtry[k], grid$nmin[k])))
tab <- xtabs(oob ~ nmin + mtry, grid)
knitr::kable(round(100 * tab, 2), caption = "OOB misclassification rate (%) by node size
 ↪  (rows) and mtry (columns).")
```

Table 3: OOB misclassification rate (%) by node size (rows) and mtry (columns).

	1	2	4	6	10	15	20
1	2.14	3.36	3.67	3.67	4.28	4.89	4.89
2	2.75	3.06	3.67	3.67	4.28	4.59	4.89
5	1.83	3.36	3.67	3.67	3.98	4.59	5.20
10	2.14	3.67	3.67	3.67	3.98	4.89	5.20
20	2.14	3.98	3.67	3.98	4.59	5.20	5.50
40	2.75	3.67	3.67	3.67	4.59	5.20	4.89

```
best <- grid[order(grid$oob, -grid$nmin, grid$mtry), ][1, ]  # ties: larger nmin first
best
#>   nmin mtry     oob
#> 3    5    1 0.01835
```

```r
set.seed(432)
fit_best <- crf(best$mtry, best$nmin, importance = TRUE)
perm_imp <- function(fit, x, y, nperm = 5) {
  oobm <- fit$ObsTrack == 0
  tree_err <- function(all) sapply(seq_along(all), function(b) {
    o <- oobm[, b]
    mean(fit$ylevels[max.col(all[[b]][o, , drop = FALSE], "first")] != y[o])
  })
  e0 <- tree_err(predict(fit, x)$AllPrediction)
  imp <- sapply(seq_len(ncol(x)), function(j) mean(replicate(nperm, {
    xp <- x; xp[, j] <- sample(xp[, j])
    mean(tree_err(predict(fit, xp)$AllPrediction) - e0)
  })))
  setNames(imp, colnames(x))
}
vi <- perm_imp(fit_best, Ztr, ytr)
c(oob_error_selected = oob_err(fit_best),
  rank_corr_with_package_VarImp = cor(vi, fit_best$VarImp[1, ], method = "spearman"))
#>            oob_error_selected rank_corr_with_package_VarImp
#>                       0.02446                       0.98496
```

```r
par(mar = c(3.2, 4.2, 0.5, 0.5), cex = 0.8)
barplot(100 * vi, las = 2, col = "grey70", ylab = "OOB error increase (pct. points)")
```

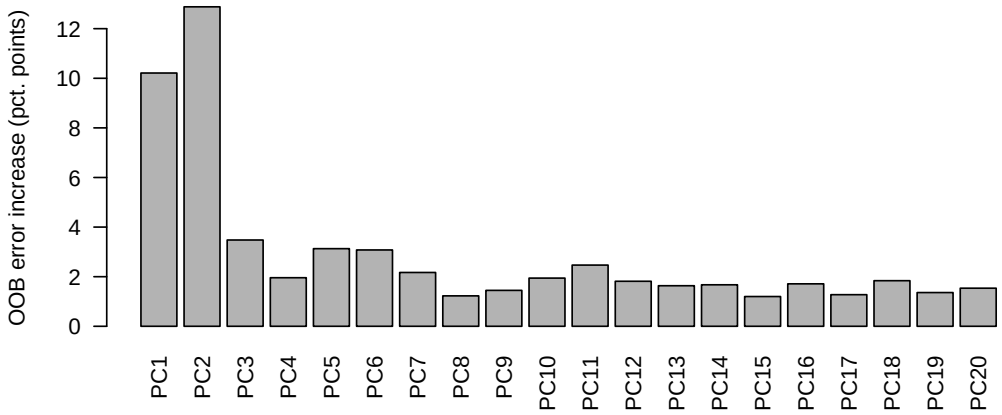


Figure 2: OOB permutation importance of the 20 principal components for the selected forest: average increase in a tree's OOB misclassification rate when the component is permuted (mean of 5 permutations).

Comments. The clearest pattern in the table is in the *columns*: the OOB error grows almost monotonically with `mtry`, from about 2% at `mtry = 1` to about 5% at `mtry = 15`–`20`, while node size hardly matters for `nmin` between 1 and 40 (the rows differ by one or two images, and with 327 training images one image is 0.31 percentage points). The selected forest is `mtry` = 1, `nmin` = 5 with OOB error 1.83%, which is the edge of the grid. This makes sense for PCA scores: the components are uncorrelated and the first few carry most of the between-digit signal, so with a large `mtry` every tree splits on PC1 and PC2 at nearly the same places and the trees become highly correlated, whereas `mtry = 1` forces each split onto a random component and produces very diverse trees that still contain the signal because several components are informative. That is the variance side of the trade-off from Question 1; the bias side is harmless here because every tree is grown deep. The newer `RLT` default `mtry = 10, nmin = 5` has OOB error 3.98%, so tuning gains about two percentage points. Permutation importance is computed as in Breiman's definition: for each tree, its OOB error is recomputed after permuting one component, and the error increases are averaged over the 1,000 trees (five independent permutations averaged). Its rank correlation with the package's own `VarImp` is 0.98, so the two definitions agree on the ordering. The importance is dominated by PC2 and PC1 (permuting either raises a tree's OOB error by 10–13 percentage points), followed by PC3, PC5 and PC6 at about 3 points; the remaining components each add 1–2.5 points and none is near zero, because with `mtry`

= 1 every component is forced into some splits of every tree. The leading components capture the gross stroke structure that separates 1 (a thin vertical stroke), 6 (a loop at the bottom) and 7 (a horizontal bar at the top), while later components encode finer, more individual variation.

(c) Test-set assessment

```
pred_te <- factor(fit_best$ylevels[predict(fit_best, Zte)$Prediction + 1],
                  levels = levels(yte))
c(test_misclassification = mean(pred_te != yte))
#> test_misclassification
#>                0.01577
table(truth = yte, predicted = pred_te)
#>      predicted
#> truth   1   6   7
#>     1 101   0   3
#>     6   0 106   0
#>     7   0   2 105
```

The test labels are used only here. The selected forest misclassifies 5 of the 317 test images (1.58%), in line with the OOB estimate of 1.83%, which confirms that the OOB rate is an honest estimate of test error and that selecting among 42 settings by OOB error did not produce a noticeably optimistic choice. In the confusion matrix every 6 is classified correctly; the errors are 3 images of a 1 predicted as 7 and 2 images of a 7 predicted as 6, the two pairs whose strokes overlap most in the leading principal components.

Question 3 (Planning a random-forest analysis skill)

(a) How I would write the skill

I would write the skill as a short contract rather than a tutorial. A capable model already knows how to fit a random forest, use out-of-bag error for tuning, compute permutation importance, and write a report, and restating those steps mostly costs tokens and occasionally pushes the model into a worse choice than its own default (for example by prescribing a grid that does not suit the data). So the standard-analysis part of the skill should be limited to the decisions that are *ambiguous* across reasonable analysts and that I want fixed for comparability: which package and what "ordinary random forest" means in it, how many trees, that the bootstrap size equals n, that OOB error is the selection criterion and the test set is touched once. Everything that is a matter of taste or of my own course requirements (report length and headings, one figure of a particular kind, how ties and one-SE-type decisions are resolved, which numbers must be read from a saved results file, what to say in a limitations paragraph) goes in a second, clearly labelled section of personal preferences, phrased as requirements, not as instructions about *how* to compute them. My experience from the SVM skill was that the model never needed the computational hand-holding but did need the preferences, and that the "ambiguities" it reported were almost all about preferences that I had left implicit. For a weaker model I would add one more layer: a checklist of the steps in order, with the expected output of each step, so it has a skeleton to follow; but I would keep that layer separate so it can be dropped when the executing AI is capable, and I would never bury the preferences inside it, because those are the instructions that cannot be inferred.

(b) Ten instructions, from most basic to most personal

1. **Inputs are given, not guessed.** Use only the data file, response, predictors and seed supplied in the request; if a variable is missing or a column has missing values, stop and report instead of improvising.
2. **Model definition.** Fit an ordinary random forest with the named package (`RLT`, reinforcement off, best-split CART trees), bootstrap samples of size n drawn with replacement, and at least 1,000 trees; state the package version and every non-default argument in the report.
3. **Choose the task type from the response.** Classification for a factor response (misclassification rate, majority vote over trees), regression for a numeric one (MSE); refuse anything else, such as survival outcomes, unless the request says so.

4. **Tuning.** Tune `mtry` and node size jointly on a small grid that scales with p and n (`mtry` around $\sqrt{p}$, $p/3$, $p/2$, p; node size from 1 to about $n/20$), selected by OOB error, with the same seed for every setting.
5. **Honest evaluation.** Hold out a test set before any modelling if the request provides one or asks for it; otherwise report the OOB error of the selected model and say explicitly that it is the only estimate; never tune on the test set.
6. **Variable importance.** Report OOB permutation importance for the selected forest (averaged over at least 5 permutations), and say in one sentence that it measures the model's reliance on a variable, not a causal effect, and that correlated predictors share importance.
7. **Tables.** One table with the OOB error of every tuning setting (rows = node size, columns = `mtry`), with the selected cell marked, plus the confusion matrix for classification; all numbers rounded to three significant digits.
8. **Figures.** Exactly one figure: a horizontal bar chart of the permutation importance (variables sorted, at most 20 shown); no caption, no title, 6×2.6 inches.
9. **Interpretation rules.** State the selected setting, its OOB error and the test error (if any), whether the best OOB error beats the runner-up by more than one OOB standard error ($\sqrt{\widehat{e}(1-\widehat{e})/n}$), and which variables dominate the importance; flag it explicitly when neighbouring settings are within OOB noise, as in Question 2.
10. **Report format.** A Quarto fragment with headings Data, Model, Tuning, Result, Limitations; at most 300 words so it fits one page at 11 pt with the figure and the tables; every number read from a results file written by a single script run; and a separate note listing any instruction in the skill that was ambiguous.